

Laboratorio 1: ¿Más procesadores significa mayor velocidad?
Curso: Computación Paralela y Distribuida
Alumno: Anthony Angel Briceño Quiroz

Descripción del problema:

Se dispone de un vector de números reales de gran tamaño. Se desea calcular la suma de todos sus elementos.

$$S = \sum A[i], i = 0, \dots, N-1$$

El programa deberá generar o inicializar el vector y calcular su suma. La solución debe producir un resultado correcto y reproducible.

Estrategia de Paralelización:

- Lo que se hizo en el código paralelizable, es poder definir una cantidad de threads en este caso podrían ser: 2,4,8,16,32 los cuales, iban a sumar, ciertos elementos del vector, este vector se tenía que partir en t threads como tal por ejemplo 16 partes cada thread agarraba una parte y a ese fragmento del vector original, lo que hacía el thread es sumar sus elementos, así los demás threads hacían lo mismo con su fragmento del vector original, para que al finalizar se guarden los resultados de esas sumas parciales en un array llamado parciales que es un array de doubles, para que después como ya son t elementos se puedan sumar secuencialmente y así obtener el valor final de la suma de N elementos ($10^8, 10^7$ o 10^6).

Parte A: Implementación Secuencial

- Genere un vector de números de tipo double.
- Implemente una versión secuencial del cálculo de la suma.
- Utilice un mecanismo de medición de tiempo apropiado, por ejemplo `std::chrono`.
- Realice varias ejecuciones para obtener mediciones representativas.

Se recomienda comenzar con $N = 10^8$ elementos y ajustar el tamaño según las características del equipo

[main.cpp](#) (Código secuencial)

```
108 numeros

80% 2026-08-19 22:25:39 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
o → ./main
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
N = 100000000
Suma = 5e+15
Tiempo transcurrido = 239222 microsegundos

107 numeros

80% 2026-08-19 22:27:33 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
o → ./main
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 7
N = 10000000
Suma = 5e+13
Tiempo transcurrido = 32546 microsegundos

106 numeros

80% 2026-08-19 22:27:42 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
o → ./main
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 6
N = 1000000
Suma = 5e+11
Tiempo transcurrido = 3261 microsegundos
```

Parte B: Identificación del paralelismo

- **¿Qué operaciones pueden realizarse independientemente?**

Las operaciones parciales del vector, osea podemos partir el vector en **M** partes y cada parte que sume por separado, es decir el pedazo 1 no depende de la suma del pedazo 2 o del 3 y así, entonces cada pedazo obtiene una suma como tal que no depende de otro pedazo existente, para este caso se hace uso de los threads, éstos pueden ir sumando esos elementos de su propio pedazo.

- **¿Qué información debe combinarse al finalizar el procesamiento?**

Al final debemos juntar la suma de todos esos **pedazos** que hicimos, como sabemos cada pedazo tiene un thread que obtiene la suma de ese pedazo, al final entonces debemos juntar esos resultados en una respuesta final como tal que seria la respuesta final, osea la suma total del vector completo.

- **¿Qué problema aparecería si varios threads modificaran directamente una misma variable de suma?**

Habría una **condición de carrera** (race condition). Si dos threads leen el valor de la variable al mismo tiempo y después escriben el resultado, uno de los dos incrementos se puede perder porque se sobrescribe el valor del otro thread. Eso hace que el resultado final sea incorrecto y ademas cambie cada vez que se ejecuta el programa.

Parte C: Implementación paralela con threads

- Implemente una versión paralela utilizando OpenMP. (threads en C++)
- Distribuya las iteraciones del ciclo entre los threads y utilice un mecanismo apropiado para obtener la suma final.

En esta primera práctica no es necesario implementar manualmente mecanismos de sincronización. El objetivo es observar el efecto del paralelismo y medirlo.

main2.cpp (Código paralelo)

Parte D: Experimento de rendimiento

- Ejecute el programa utilizando diferentes cantidades de threads. Considere como mínimo los valores razonables para el equipo utilizado.

Threads	Tiempo (s)	Speedup	Eficiencia
1	0.231434	1.0	1.0
2	0.117892	1.963098	0.981549
4	0.061336	3.773239	0.943310
8	0.034084	6.790192	0.848774
16	0.023841	9.707577	0.606724
32	0.021157	10.938603	0.341831

- Prueba hecha con cantidad de elementos = 10^8
- Speedup: $S_p = T_1 / T_p$
- Eficiencia: $E_p = S_p / p$
- Prueba ejecutada 10 veces, para luego obtener la media de dichos resultados.

[pruebasSecuenciales.py](#)

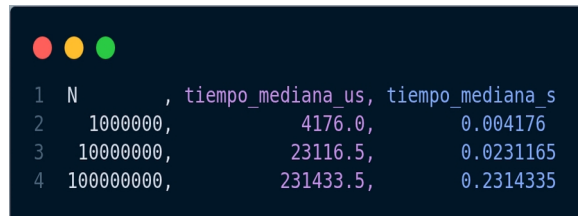
[pruebasParalelas.py](#)

Parte E: Influencia del tamaño del problema

- Repita el experimento para diferentes tamaños del vector. Se recomienda utilizar:
 - $N = 10^6$
 - $N = 10^7$
 - $N = 10^8$

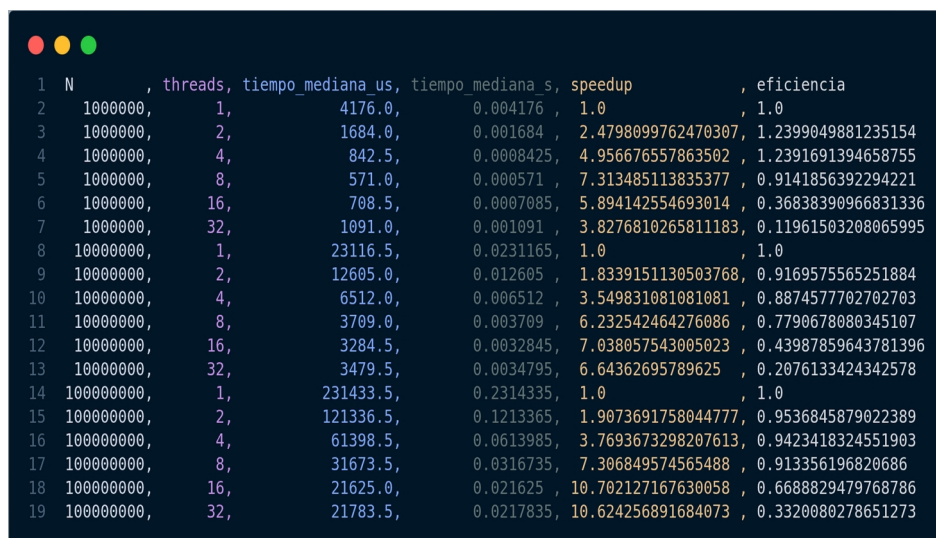
Compare cómo cambia el speedup cuando aumenta el tamaño del problema. Si el equipo no dispone de memoria suficiente, seleccione valores adecuados y registre la decisión

Resultados Secuenciales:



1	N	, tiempo_mediana_us,	tiempo_mediana_s
2	1000000,	4176.0,	0.004176
3	10000000,	23116.5,	0.0231165
4	100000000,	231433.5,	0.2314335

Resultados Paralelos:

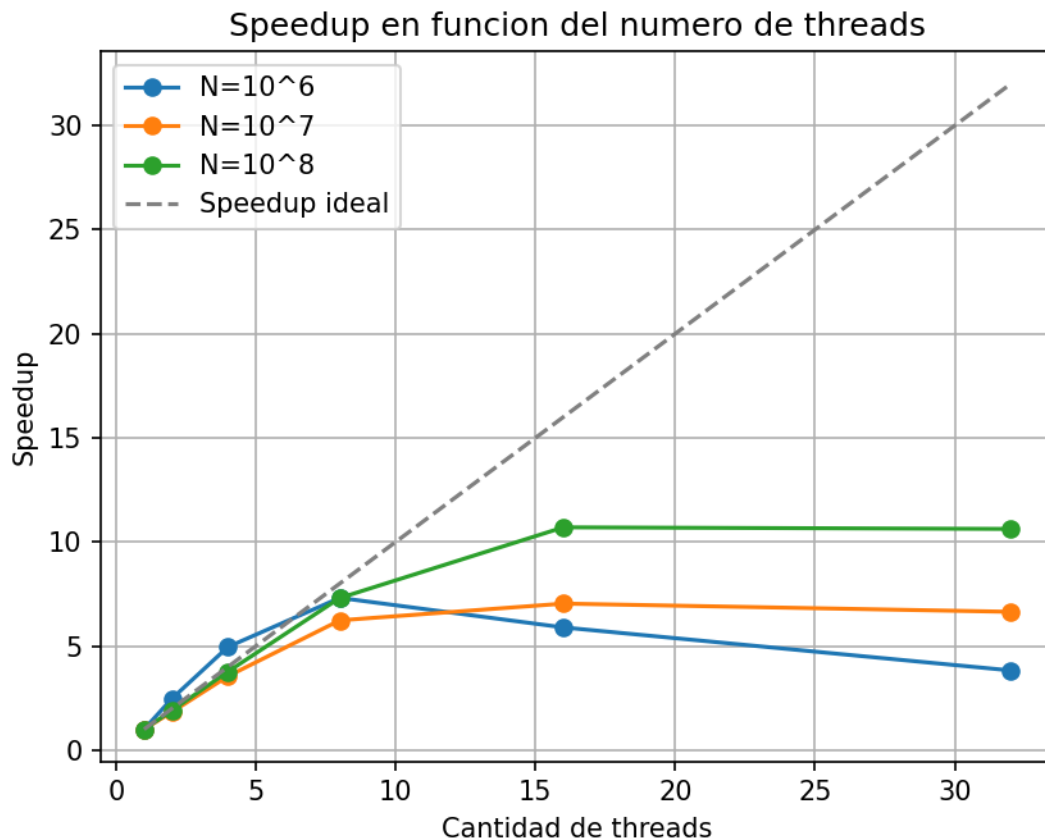


1	N	, threads,	tiempo_mediana_us,	tiempo_mediana_s,	speedup	, eficiencia
2	1000000,	1,	4176.0,	0.004176,	1.0	1.0
3	1000000,	2,	1684.0,	0.001684,	2.4798099762470307,	1.2399049881235154
4	1000000,	4,	842.5,	0.0008425,	4.956676557863502,	1.2391691394658755
5	1000000,	8,	571.0,	0.000571,	7.313485113835377,	0.9141856392294221
6	1000000,	16,	708.5,	0.0007085,	5.894142554693014,	0.36838390966831336
7	1000000,	32,	1091.0,	0.001091,	3.8276810265811183,	0.11961503208065995
8	10000000,	1,	23116.5,	0.0231165,	1.0	1.0
9	10000000,	2,	12605.0,	0.012605,	1.8339151130503768,	0.9169575565251884
10	10000000,	4,	6512.0,	0.006512,	3.549831081081081,	0.8874577702702703
11	10000000,	8,	3709.0,	0.003709,	6.232542464276086,	0.7790678080345107
12	10000000,	16,	3284.5,	0.0032845,	7.038057543005023,	0.43987859643781396
13	10000000,	32,	3479.5,	0.0034795,	6.64362695789625,	0.2076133424342578
14	100000000,	1,	231433.5,	0.2314335,	1.0	1.0
15	100000000,	2,	121336.5,	0.1213365,	1.9073691758044777,	0.9536845879022389
16	100000000,	4,	61398.5,	0.0613985,	3.7693673298207613,	0.9423418324551903
17	100000000,	8,	31673.5,	0.0316735,	7.306849574565488,	0.913356196820686
18	100000000,	16,	21625.0,	0.021625,	10.702127167630058,	0.6688829479768786
19	100000000,	32,	21783.5,	0.0217835,	10.624256891684073,	0.3320080278651273

¿Cómo cambia el speedup?

El speedup como tal va cambiando de acuerdo a la cantidad de elementos del vector, en este caso se puede observar el mayor speedup se generó cuando se corrieron 16 threads, porque en este prueba el programa corrió **10.7** veces mas rapido que el programa secuencial, si nos percatamos también, el que le sigue es el de 32 threads que corrió **10.62** veces mas rápido que el programa secuencial, esto tiene que ver con el tema de el overhead, es decir no siempre mas threads va a ganar, ya que la creación de un thread involucra un costo real (llamada al kernel, stack, etc). Osea con 16 threads es el pico máximo de paralelismo, usar 32 sería algo que no vale la pena y que seria desventajoso para nosotros, ya que suma overhead y no se reparte de forma eficiente el trabajo. Esto tiene mucho que ver con la cantidad de cores como tal fisicos, ya que en mi caso la pc en la que se ha hecho pruebas existe 32 procesadores lógicos, de los cuales 16 son físicos (2 hilos por núcleo)

Gráfica de speedup en función del número de threads.



Preguntas de análisis:

1. ¿Qué parte del algoritmo puede ejecutarse en paralelo?

El ciclo de las sumas de los elementos, ya que si hacemos subconjuntos del conjunto de numeros inicial, la suma de esos subconjuntos no depende de la suma de otro subconjunto, entonces podemos obtener tantas sumas como subconjuntos hayamos creado luego, si nos percatamos, la cantidad de sumas que hay es una cantidad que se puede hacer de manera mucho mas rapido que sumar los N valores del vector.

2. ¿Qué problema aparece al calcular la suma utilizando múltiples threads?

El problema mas común seria el de leer y escribir una variable a la cual todos los threads tengan acceso, esto se conoce como Race Condition, lo que causaría que se pierda un resultado y no sea el adecuado, osea chancar un resultado y asi perder incrementos.

3. ¿El speedup obtenido es lineal? Justifique utilizando los resultados experimentales.

No, el speedup no es lineal, lo que se se busca seria lo sgte: $Sp = p$, osea si es que tenemos 2 threads que sea el doble de rápido, si tenemos 8 threads que sea 8 veces más rápido y así sucesivamente hasta por ejemplo 32 threads, 32 veces mas rápido, pero esto en las pruebas no se logra, porque si vemos los resultados; observamos que para 2 threads se logra un speedup de 1.91 lo cual es casi 2, si pasamos a 4 threads no da 3.77, si pasamos a 8 threads nos da 7.31 cada vez ese speedup ideal va bajando por decimales por ejemplo si tenemos 16 threads nos da un speedup de 10.70 el cual en las pruebas que hemos hecho es le mejor de todos superando al de 32 threads por 0.08

y si lo vemos en tema de porcentajes solo se estaría llegando al mejor resultado en un 67% con 16 threads osea 10.70 de 16.

4. ¿A partir de qué cantidad de threads el rendimiento deja de mejorar significativamente?

Bueno se ve que a partir 16 threads el rendimiento deja de mejorar, osea de 1 a 2 vemos una ganancia de 91%, de 2 a 4 vemos una ganancia de 97%, de 4 a 8 vemos una de 94% pero de 8 a 16 esa ganancia que debería aumentar en algo, disminuye considerablemente y cae a un 46% y mas aun si vemos de 16 a 32 que baja en un 1%, ese es en la prueba con 10^8 elementos en la prueba de 10^6 decae con mucha mas facilidad a un 5.89 de speedup en 16 threads y un 3.83 en 32 threads, por lo que se puede deducir que el tamaño de los datos si influye directamente. Pero para dar una respuesta mas general, afirmo que existe un punto en donde el rendimiento deja de mejorar y esto depende del tamaño del problema, osea si agregamos mas threads no aportamos algo más, si no que perjudicamos levemente el problema.

5. ¿Qué ocurre cuando se utilizan más threads que núcleos disponibles?

Cuando estuve probando con más threads que los núcleos disponibles, me percate que si ponemos una cantidad cercana a la cantidad de nucleos disponibles de la pc, tiende a mantener la misma cantidad de tiempo para dar la suma total, sin embargo esto como tal no asegura que el speedup sea mayor o mejor que las pruebas que se han hecho cuando se probaba con la cantidad de threads menor a los cores de la pc.

Estuve haciendo pruebas manuales con con cantidad de threads incluso altas como: 240, 400, 600, 1200 y el tiempo en el que se demoraba en dar la respuesta era superior al de las otras pruebas, lo que indica que mas threads en este caso no llega a ser la mejor opción

```
80% 2026-08-20 11:50:10 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 64
cantidad_numeros = 100000000
Threads = 64
Suma = 5e+15
Tiempo transcurrido = 18390 microsegundos
```

```
80% 2026-08-20 11:50:19 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 120
cantidad_numeros = 100000000
Threads = 120
Suma = 5e+15
Tiempo transcurrido = 17973 microsegundos
```

```
80% 2026-08-20 11:50:44 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 240
cantidad_numeros = 100000000
Threads = 240
Suma = 5e+15
Tiempo transcurrido = 19921 microsegundos
```

```
80% 2026-08-20 11:51:11 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 400
cantidad_numeros = 100000000
Threads = 400
Suma = 5e+15
Tiempo transcurrido = 22689 microsegundos
```

```
80% 2026-08-20 11:51:27 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 600
cantidad_numeros = 100000000
Threads = 600
Suma = 5e+15
Tiempo transcurrido = 27996 microsegundos
```

```
80% 2026-08-20 11:51:45 AnthonyDesktop in ~/Documentos/Semestre 2026-2/Paralela/labs/lab1
± |main U:6 x| → ./main2
Ingrese el valor del exponente para calcular la cantidad de numeros del vector: 8
Ingrese la cantidad de threads a usar (2,4,8,16,32): 1200
cantidad_numeros = 100000000
Threads = 1200
Suma = 5e+15
Tiempo transcurrido = 41978 microsegundos
```

6. ¿El tamaño del problema influye en el speedup obtenido? Explique.

Si, el tamaño del problema si influye en el speedup obtenido, nos percatamos en lo sgte: cuando hicimos pruebas con la cantidad de 10^6 elementos en el vector, con 16 threads vimos que el speedup fue de 5.89, con 10^7 elementos el speedup con 16 threads fue de 7.04 y con 10^8 el speedup fue de 10.7; esto quiere decir si tenemos un mayor N el trabajo del thread es mas grande en relación a la creación del mismo y la coordinación de éste con los demas, pero en un N menor hace que el overhead pese más que el mismo trabajo del thread, esto da como respuesta que el speedup se degrade más rápido a medida que aumentan los threads teniendo un speedup de 3.82 con 32 threads.

7. Proponga al menos dos razones que podrían explicar por qué el speedup no aumenta proporcionalmente al número de threads.

- El overhead de creación y sincronización de los threads, osea puede resultar mas costoso la creación del mismo thread que el esfuerzo que haga él con la tarea que se le asigne.
- Si nosotros sumamos un vector, esta suma depende digamoslo asi de la cantidad de memoria que tengamos en nuestra pc (RAM → caché) , si tenemos más threads leyendo en simultáneo, se llegaría a saturar el bus antes de que el paralelismo extra aporte un beneficio como tal.

Datos del equipo:

```
80% 2026-08-20 12:27:28 AnthonyDesktop in ~
o → lscpu | grep -E "Model name|Socket|Núcleo|Hilo|CPU(s)|MHz|Arquitectura"; echo "---RAM---"; free -h
Arquitectura:          x86_64
CPU(s):                32
Lista de la(s) CPU(s) en línea:      0-31
Hilo(s) de procesamiento por núcleo: 2
Núcleo(s) por «socket»:      16
«Socket(s)»:           1
CPU(s) factor de escala MHz:      48%
CPU MHz máx.:             5462,7109
CPU MHz min.:             427,8030
CPU(s) del nodo NUMA 0:         0-31
---RAM---
              total        usado      libre  compartido  búf/caché  disponible
Mem:          30Gi         8,1Gi        11Gi        102Mi        11Gi        22Gi
Inter:         8,0Gi          0B         8,0Gi
80% 2026-08-20 12:27:29 AnthonyDesktop in ~
o → grep "model name" /proc/cpuinfo | head -1
model name      : AMD Ryzen 9 8945HX with Radeon Graphics
```

Conclusión: ¿más threads significan necesariamente mayor velocidad?

No, mas threads no van a significar mayor velocidad, como hemos visto en incluso he llegado a probar con 1200 threads y el tiempo que le ha tomado a la pc dar la respuesta es mucho mayor que otros resultados con menos threads.

Por lo que se debe analizar bien, el problema, la cantidad de datos y hacer pruebas para poder determinar si una cantidad de threads es la mejor para dicho problema.